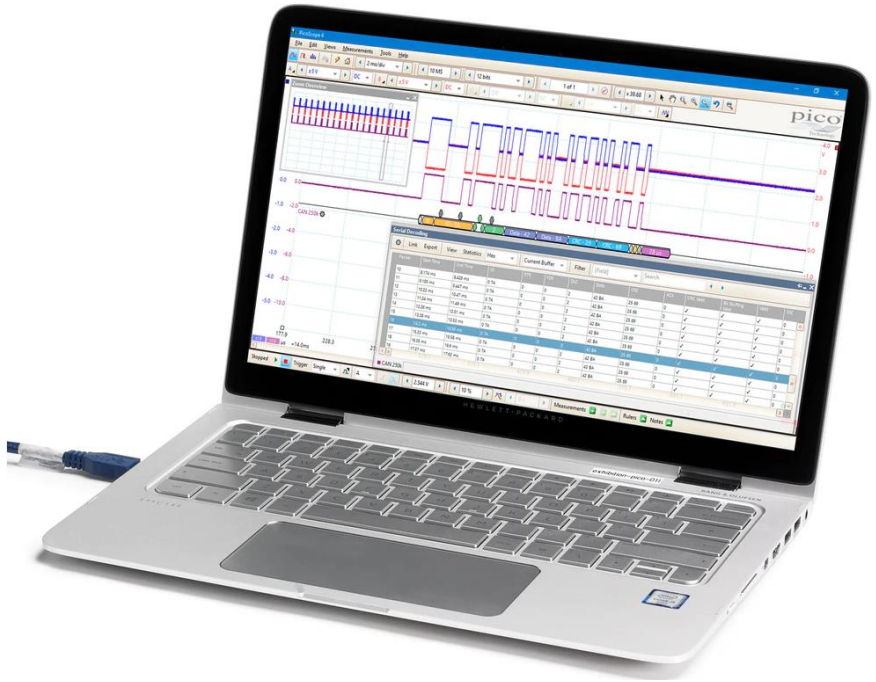
Exercise

Telemetry &
Telecommand

UART

(HAL_UART)



• Tasks:

2.1 Telemetry & Telecommand:

Hint

- In order to avoid defining two HALs for the same interface, you can use the **uart_stdout** HAL interface at your project as following :

```
namespace RODOS {extern HAL_UART uart_stdout;}
#define TeleUART uart_stdout
```

a. Implement a project in which you can receive Telecommands through UART3 with enabled DMA with following requirements:

• The Telecommand thread should be suspended until new data are received at UART3.

• The message body of the Telecommand should have the following format "$<X><data>#" where:
"$" is the Message Start Character [1 byte].
X is a predefined Message ID Character [1 byte].
data are ASCII characters which correspond to a valid floating point number [0 –10 bytes].
"#" is the Message End Character [1 byte].

$T1#	Telemetry on
$T0#	Telemetry off

• Decoding the message should only complete if correct message format is received. Otherwise, the message should discarded and an error message "Message Corrupted" should be send through UART3.

• The message should discarded and an error message "Message Timeout" should send through UART3 if a complete message is not received within a time window of 5000 milliseconds.

Hint

- You can use **suspendUntilDataReady()** function to suspend the Telecommand thread until new data are arrived through UART3.

• Tasks:

b. Create another two threads in the project that can perform the following:

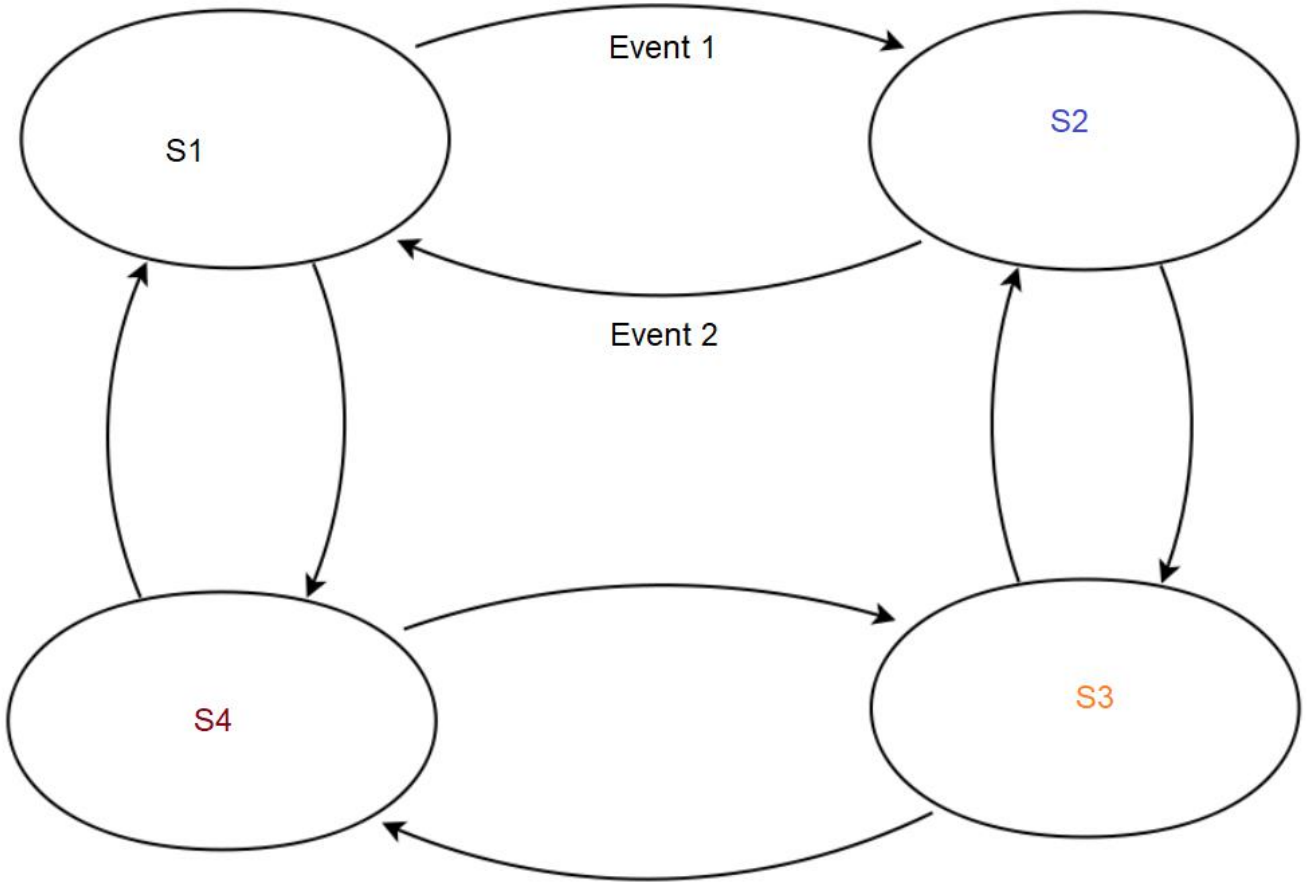
- Thread 1 running at t1 time interval: Increment or Decrement a counter where Counter=Counter+StepSize and Toggle Red LED.

- Thread 2 running at t2 time interval: Send Telemetry data through UART3 and Toggle Blue LED.

- **t1 and t2 are the thread time intervals in milliseconds and should be able to update through Telecommands.**
- **StepSize is the increment or decrement step size for the counter and should be able to update through Telecommand.**
- **Any thread should not be suspended forever if its corresponding time interval set to zero.**
- **Counter, StepSize and actual running time in seconds are the telemetry data need to be sent.**

Tips:

- Design and implement **State-Machine** to decode the Telecommands.
- You can send telemetry data through UART3 with the help of **sprintf** function that write formatted data to string.
- **suspendCallerUntil(NOW()+t1*MILLISECONDS)** function will suspend the thread for t1 milliseconds period.
- Use **Topics** to **exchange data** between the threads.
- Standard RODOS **PRINTF** function is used to print data through a predefined **uart_stdout** HAL Interface. This is by default defined as UART3 at 115200 baud rate. In order to change these default setting, You can change the **UART_DEBUG** definition in **params.h** at rodos>>src>>bare-metal>>stm32f4.


Event 1
S1
S2
Event 2
S4
S3